

ФГАОУ ВО «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №3

Принципиальная схема процессора RISC-IV
(по дисциплине «Архитектура компьютера»)

Выполнил: Старченко Александр
Николаевич,
АРХКОМП 3.3, Р3231, 467586.

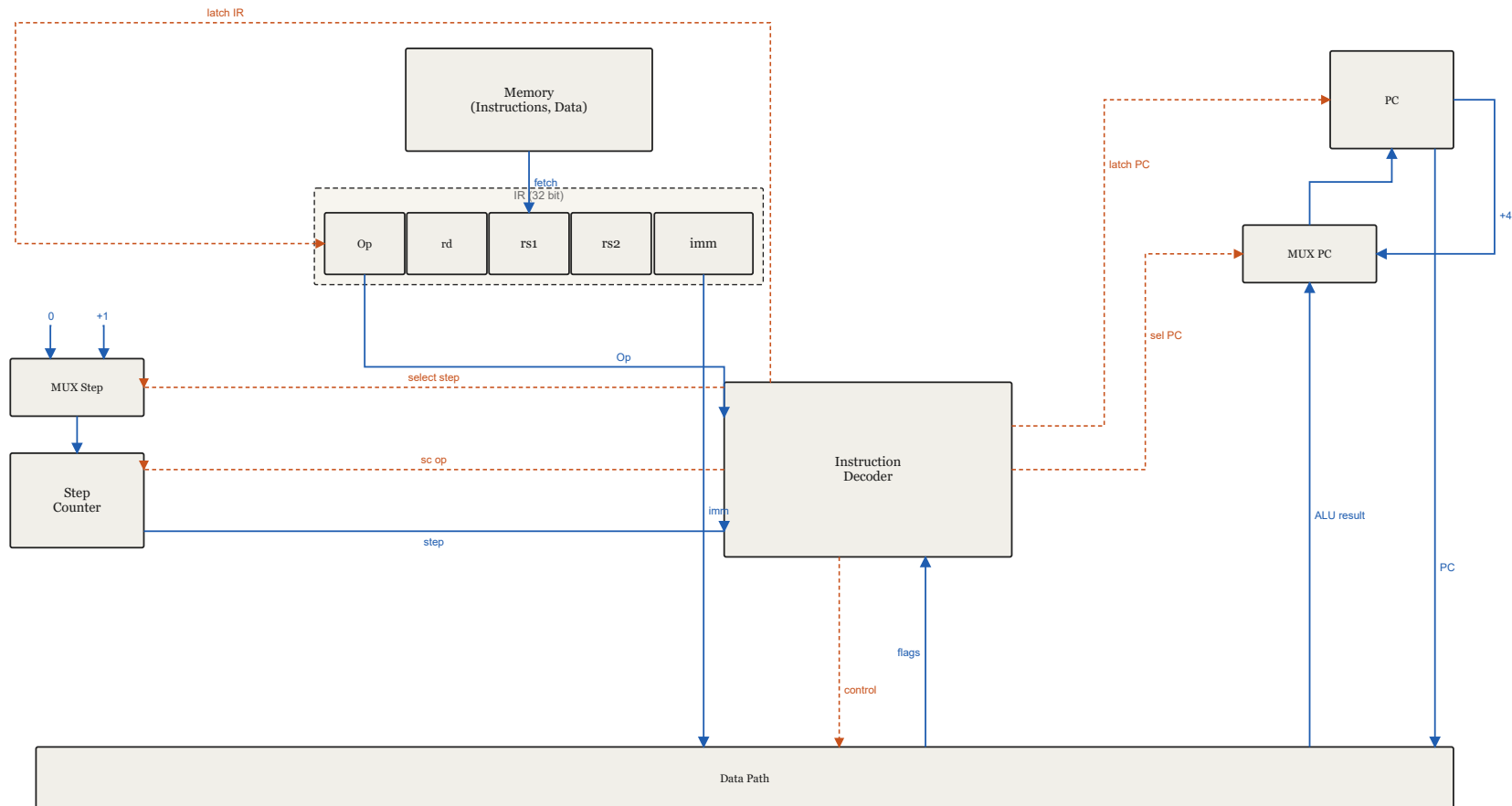
Проверил: Пенской Александр
Владимирович

г. Санкт-Петербург, Россия
2026

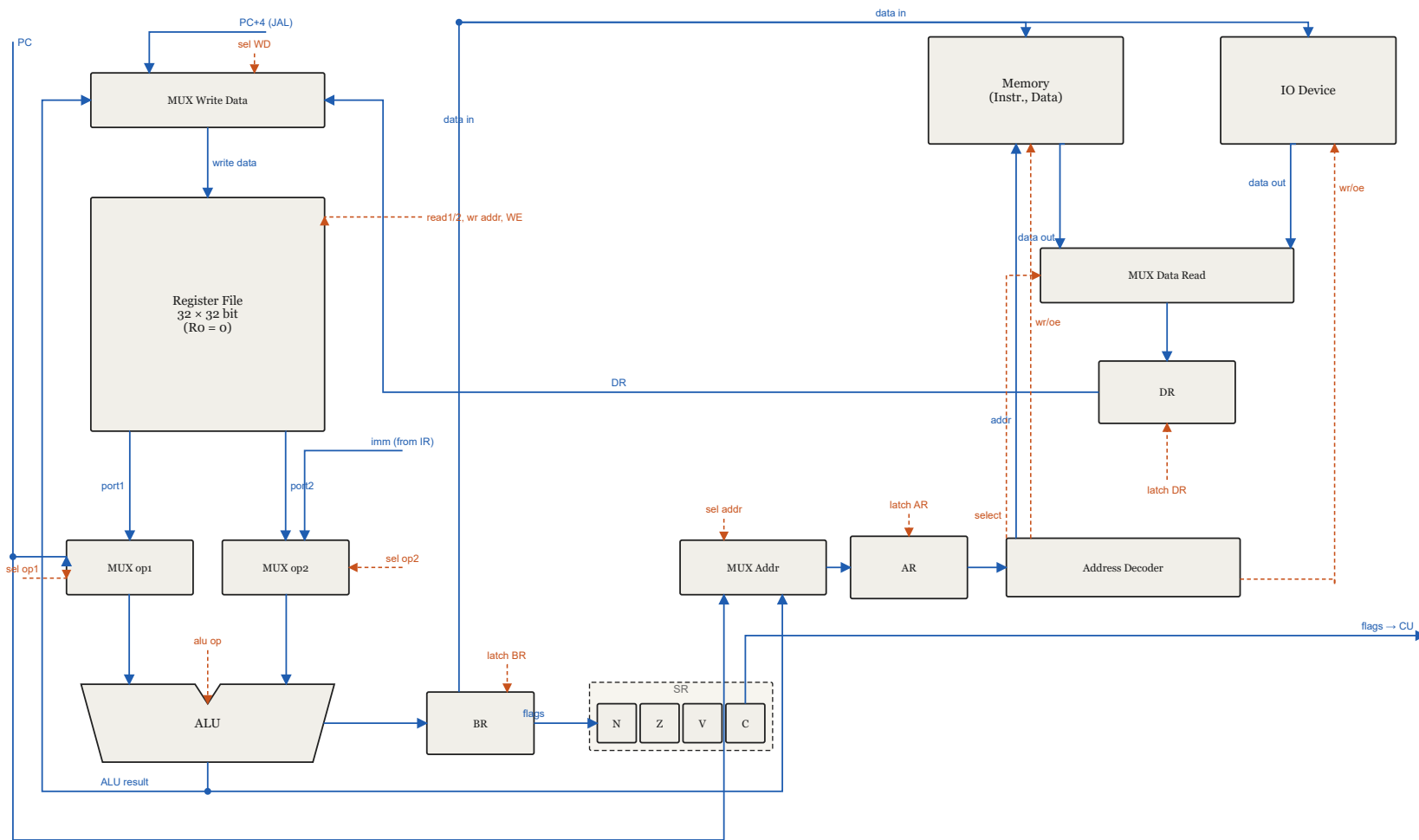
Содержание

1. Control Unit
2. Data Path
3. Пояснения

1. Control Unit



2. Data Path



3. Пояснения

1. **Архитектура.** RISC-IV — учебная 32-битная RISC-архитектура типа *load-store*. Длина инструкции фиксирована и равна 4 байтам, формат инструкции $\langle \text{Op} \mid \text{rd} \mid \text{rs1} \mid \text{rs2} \mid \text{imm} \rangle$. Все арифметические и логические операции выполняются исключительно над регистрами; обращение к памяти реализуется только инструкциями *lw* и *sw*.
2. **Register File.** Содержит 32 регистра по 32 бита. Имеет *два порта чтения* (*port1*, *port2*) и *один порт записи*; адресуется полями *rs1*, *rs2*, *rd* из IR. Регистр *R0* аппаратно занулён: чтение всегда возвращает 0, запись игнорируется.
3. **Control Unit.** Состоит из Step Counter и Instruction Decodera. Step Counter хранит номер текущего такта внутри инструкции; MUX Step перед ним выбирает между значениями 0 (сброс при переходе к новой инструкции) и +1 (инкремент такта). Декодер по полю *Op* и значению Step Counter формирует все управляющие сигналы (на схеме — пунктирные шины).
4. **Программный счётчик.** PC обновляется через MUX PC, выбирающий между PC+4 (нормальный шаг) и результатом ALU (цель условного или безусловного перехода). Сигнал *latch PC* разрешает запись нового значения.
5. **Память и шины.** Память и устройства ввода-вывода живут в едином адресном пространстве (*memory-mapped I/O*). Address Decoder по старшим битам адреса из AR определяет, какому устройству адресовано обращение, и формирует соответствующие *wr/oe*, а также сигнал *select* для MUX Data Read.
6. **Тракт чтения данных.** Прочитанные значения с шины данных попадают в MUX Data Read (выбирающий между Memory и IO), далее защёлкиваются в DR. Из DR значение через MUX Write Data записывается обратно в Register File.
7. **Тракт записи и ALU.** На входы ALU подаются выходы MUX op1 и MUX op2: левый вход выбирает между *rs1* и PC (для *auipc/jal*), правый — между *rs2* и *imm* (для I-типа). Результат защёлкивается в BR, откуда идёт в Memory/IO (для *sw*) или через MUX Write Data — обратно в RF.
8. **Адресация памяти.** Источник адреса выбирается MUX Addr: либо PC (фаза *fetch*), либо результат ALU (для *lw/sw*, где $\text{адрес} = \text{rs1} + \text{imm}$). Адрес защёлкивается в AR и подаётся в Address Decoder.
9. **Флаги.** По итогу операции ALU формируются флаги N, Z, V, C, которые защёлкиваются в SR и подаются в Instruction Decoder — декодер использует их для разрешения условных переходов.

10. **Стиль шин.** Сплошные линии со стрелками — шины данных (*data bus*); пунктирные — управляющие сигналы (*control bus*). Тонкая пунктирная рамка вокруг IR и SR обозначает их как составные регистры из нескольких полей.